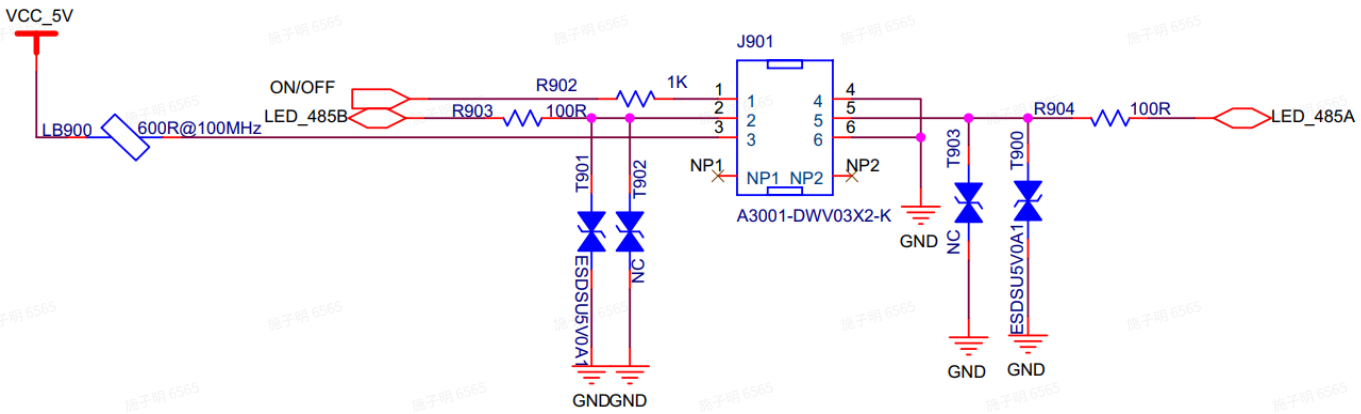
R3协议接口说明

1. CCB<->BLE

1.1 硬件接口

1. 接口原理图



2. 接口说明

序号	功能说明
1	电源按键引脚(按键被按下电平被拉低)
2	TX(Slave)
3	5V电源
4	GND
5	RX(Slave)
6	GND

1.2 通信协议

1.2.1 通信接口

UART，通信速率460800，1起始位，1停止位，无校验。

1.2.2 协议接口说明

1. 数据解析

ppx_packet_status_t ppx_com_ble_parse(IN uint8_t *pdata, IN uint8_t data_len, INOUT ppx_ble_msg_t *ble_msg);

参数说明

uint8_t *pdata　　串口接收到的数据

data_len　　串口接收到的数据长度

ppx_ble_msg_t *ble_msg　解包信息

返回值 1为解析通过，非1为解析失败

2. 数据组包

uint16_t ppx_com_ble_format(IN ppx_cmd_type_t cmd_type, IN ppx_ble_msg_t *ble_msg, OUT void *buffer);

参数说明

ppx_cmd_type_t cmd_type　组包类型 （Master请求/Slave响应）

ppx_ble_msg_t *ble_msg　组包信息

void *buffer　　组包输出

返回值　　组包输出的数据长度

3. 协议接口使用

协议中定义了 ppx_ble_data_t g_ppx_ble_data全局变量，每个结构体成员有对应的寄存器地址ppx_ble_reg_t，发送端先填充ppx_ble_msg_t内容(如果需要执行写操作，先修改全局变量内容)，确定执行的范围，最后调用ppx_com_ble_format组包，组包生成的内容通过串口发出；收到来自串口的数据调用ppx_com_ble_parse解析，解析成功后会更新全局变量内容。

4. 示例

点亮LED并显示100

Master

修改g_ppx_ble_data中 led_msg成员的内容

g_ppx_ble_data.led_msg.screen_on = 1;

g_ppx_ble_data.led_msg.brightness = 0;

g_ppx_ble_data.led_msg.digital = 100;

定义ppx_ble_msg_t ble_msg并填充内容

ble_msg.id = PPX_ID_BLE;

ble_msg.cmd = PPX_MSG_WRITE;

ble_msg.reg_addr = PPX_BLE_LED_MSG_REG;

ble_msg.reg_nums = 1;

将ble_msg作为组包参数调用ppx_com_ble_format生成数据包并通过串口发出。

Slave

定义ppx_ble_msg_t ble_msg，将接收到的数据及ble_msg作为参数调用ppx_com_ble_parse，解析通过后Slave端的g_ppx_ble_data被修改，ble_msg内容可看出被修改的范围。

1.3 数据结构定义 （详细内容参考ppx_ble.h）

```
/* define struct of ppx ble msg */
typedef struct
{
    uint8_t id;          /* dev id */
    uint8_t cmd;         /* write/read cmd */
    uint8_t reg_addr;    /* ble data addr*/
    uint8_t reg_nums;    /* ble reg number */
} ppx_ble_msg_t;

/* define struct of ppx led msg 32bit */
typedef struct
{
    uint32_t screen_on    : 1;  // Display switch: 1 on, 0 off
    uint32_t brightness   : 3;  // Brightness level 0-7
    uint32_t blink_period : 3;  // Blink period: N * 200ms
    uint32_t blink_duty   : 4;  // Blink duty cycle: (N + 1) / 16 * blink_period
    uint32_t blink_en     : 5;  // Blink enable: bit0-bit4 for digital, percent, charge, gear, blink status
    uint32_t color_flag   : 1;  // Color flag 0 white, 1 orange
    uint32_t err_flag     : 2;  // Error code flag 0 no error, 1 E, 2 L, 3 R
    uint32_t digital      : 8;  // digital bit0-8: 0-188
    uint32_t percent      : 1;  // percent: 0 off, 1 white
    uint32_t charge       : 1;  // charge: 0 off, 1 green
    uint32_t gear         : 2;  // gear: 0 off, 1 D1, 2 D2, 3 D3
    uint32_t light        : 1;  // light: 0 off, 1 on
```

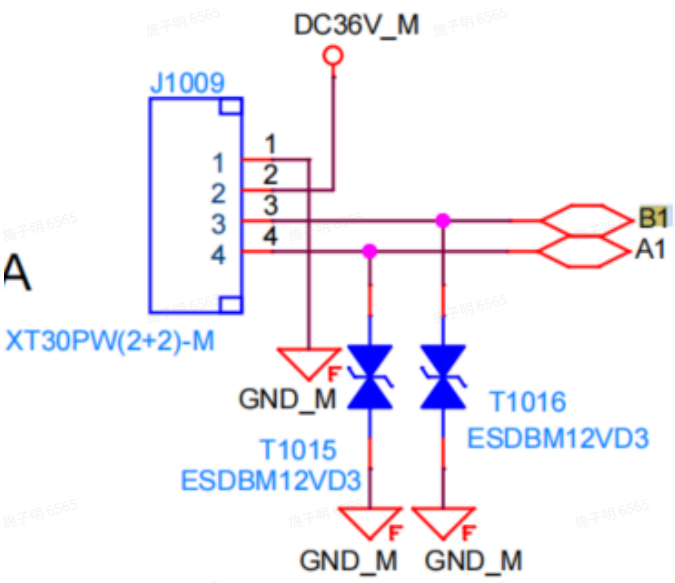
```
} ppx_led_msg_t;

typedef struct
{
    uint8_t       id_num;             // Device ID number
    uint8_t       model[PPX_MODEL_SIZE];  // Model (8 bytes)
    uint8_t       serial_num[PPX_SN_SIZE];// Serial number (26 bytes)
    uint8_t       hw_version;         // Hardware version
    uint8_t       sw_version[PPX_SW_VER_SIZE]; // Software version (20 bytes)
    uint16_t      ldr_value;          // light-dependent resistor brightness
    uint32_t      status;             // @ref ppx_ble_status_t
    int8_t        joystick_x;         // Joystick X value -100 ~ 100
    int8_t        joystick_y;         // Joystick Y value -100 ~ 100
    int16_t       joystick_x_adc;     // Joystick X ADC value (mV)
    int16_t       joystick_y_adc;     // Joystick Y ADC value (mV)
    ppx_led_msg_t  led_msg;           // led display message
    uint32_t      voice_data;         // Voice data bit0-bit15 : voice num; bit16-bit31 : volume
    uint32_t      dat_setting;        // @ref ppx_ble_data_setting_t
} ppx_ble_data_t;

/* define enum of ppx protocl data ble reg addr  */
typedef enum
{
    PPX_BLE_ID_NUM_REG     = 0,  /* 0x00 */
    PPX_BLE_MODEL_REG            ,
    PPX_BLE_SERIAL_NUM_REG  = 2,  /* 0x02 */
    PPX_BLE_HW_VERSION_REG      ,
    PPX_BLE_SW_VESRION_REG      ,
    PPX_BLE_LDR_VALUE_REG    = 5,  /* 0x05 */
    PPX_BLE_STATUS_REG           ,
    PPX_BLE_JOYSTICK_X_REG       ,
```

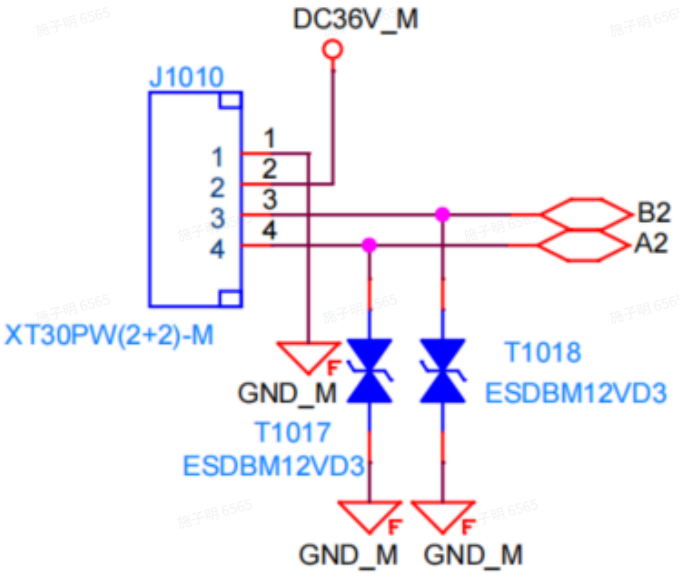
```
    PPX_BLE_JOYSTICK_Y_REG      ,
    PPX_BLE_JOYSTICK_X_ADC_REG ,
    PPX_BLE_JOYSTICK_Y_ADC_REG ,
    PPX_BLE_LED_MSG_REG    = 11, /* 0x0B */
    PPX_BLE_VOICE_DATA_REG      ,
    PPX_BLE_DAT_SETTING_REG    ,
    PPX_BLE_MAX_REG
} ppx_ble_reg_t;
```

2. CCB<->MCB

2.1 硬件接口

1. 接口原理图





2. 接口说明

序号	功能说明
1	GND(电池负极)
2	电池正极
3	RS485-B

4	RS485-A

2.2 通信协议

2.2.1 通信接口

RS485，通信速率460800，1起始位，1停止位，无校验。

2.2.2 协议接口说明

1. 数据解析

ppx_packet_status_t ppx_com_region_parse(IN uint8_t *pdata, IN uint16_t data_len, INOUT ppx_region_ctrl_t *region_ctrl);

参数说明

uint8_t *pdata　　串口接收到的数据

data_len　　串口接收到的数据长度

ppx_region_ctrl_t *region_ctrl　　解包信息

返回值 1为解析通过，非1为解析失败

2. 数据组包

uint16_t ppx_com_region_format(IN ppx_cmd_type_t cmd_type, IN ppx_region_ctrl_t *region_ctrl, OUT void *buffer);

参数说明

ppx_cmd_type_t cmd_type　　组包类型 （Master请求/Slave响应）

ppx_region_ctrl_t *region_ctrl　　组包信息

void *buffer　　组包输出

返回值　　组包输出的数据长度

3. 协议接口使用

使用时需自定义ppx_region_ctrl_t类型数据，ppx_region_ctrl_t包含msg和data，msg是组包信息，data是数据内同(如果需要执行写操作，先修改全局变量内容)，确定执行的范围，最后调用ppx_com_region_format组包，组包生成的内容通过串口发出；收到来自串口的数据调用ppx_com_region_parse解析，解析成功后会更新region_ctrl.data的内容。

2.3 数据结构定义 （详细内容参考ppx_region.h）

/* define struct of ppx region msg */

typedef struct

```
{
    uint8_t parse_status;
    uint8_t cmd_status;
    uint8_t data_status;
} ppx_region_excp_t;


/* define struct of ppx region msg */
typedef struct
{
    uint8_t id;    /* dev id /
    uint8_t cmd;   /write/read cmd */

    uint8_t msg_type;   /* for master used /
    uint8_t reg_addr;   /region data addr*/
    uint8_t reg_nums;   /* region data number */

    ppx_region_excp_t reg_excp; /* exception response */
} ppx_region_msg_t;


/* define ppx_region data struct */
#pragma pack (1)
typedef struct
{
    /* ppx_region read data  */
    uint8_t   id_num;                    //ID
    uint8_t   model[PPX_MODEL_SIZE];

    uint8_t   serial_num[PPX_SN_SIZE];
    uint16_t  hw_version;
```

```
uint8_t  sw_version[PPX_SW_VER_SIZE];

uint32_t mcu_errcode;    /* err code */
int16_t  motor_speed;    /fb speed */

uint16_t bus_voltage;    /* 0.1V */
uint16_t bus_current;    /0.1A */

uint8_t  rim_state;
uint8_t  ctrl_model;
int16_t  speed_ref;      /* rpm */

int16_t  mosfet_temp;    /* deg */
int16_t  motor_temp;     /deg */

uint8_t  motor_limit_flg;
int32_t  motor_vq;
int16_t  motor_iq;

uint8_t  motor_cali_state;
uint16_t motor_cali_res;  /* mΩ */
uint16_t motor_cali_ld;   /uH */
uint16_t motor_cali_lq;   /uH */
uint16_t motor_cali_bemf; /uV/rpm */

uint8_t  brake_state;    /* ppx_brake_state_t */
uint32_t single_mileage;  /m */

/* ppx_region  write data */
uint16_t rt_setting;
```

```
    uint8_t  run_mode;      /* ppx_run_mode_t */
    uint8_t  road_cond;     /deg */

    int16_t  target_speed;   /* rpm */
    uint16_t target_accel;   /m/s */
    int16_t  target_current; /0.1A */

    uint16_t rated_voltage;   /* 0.1V */
    uint16_t rated_current;   /0.1A */

    uint32_t dat_setting;    /* data cfg */

    uint32_t reserved_data;   /* reserved */
} ppx_region_data_t;
#pragma pack()


/* define ppx_region control struct */
typedef struct
{
    ppx_region_msg_t   msg;    /* message info */
    ppx_region_data_t  data;   /register data */
} ppx_region_ctrl_t;
```

3. 公共数据结构

```
/* define ppx project information */
#define PPX_SW_VER_SIZE          ( 32 )
#define PPX_VER_MIN_SIZE         ( 12 )
#define PPX_MODEL_SIZE           ( 8 )
#define PPX_SN_SIZE              ( 26 )
#define PPX_BIN_IAP_VER_OFFSET     (0x0800)   /* BIN IAP offset 2K */
```

```
#define PPX_BIN_APP_VER_OFFSET     (0x2800)    /* BIN APP offset 10K */

/* define ppx protocl frame head */
#define PPX_FRAME_HEAD            ( 0xA5 )
#define PPX_FRAME_END             ( 0x55 )
#define PPX_DATA_TAG              ( 0x33 )
#define PPX_DATA_REPHEAD_H        ( 0xAB )  /* 0xA5 -> 0xABBA */
#define PPX_DATA_REPHEAD_L        ( 0xBA )  /* 0xA5 -> 0xABBA */
#define PPX_DATA_REPEND_H         ( 0xCD )  /* 0x55 -> 0xCDDC */
#define PPX_DATA_REPEND_L         ( 0xDC )  /* 0x55 -> 0xCDDC */
#define PPX_DATA_REPHEAD_2        ( 0xBB )  /* 0xABBA -> 0xABBB BA */
#define PPX_DATA_REPEND_2         ( 0xDD )  /* 0xCDDC -> 0xCDDD DC*/

/* define ppx return status */
typedef enum
{
   PPX_ERROR = -1,
   PPX_FALSE = 0,
   PPX_TRUE  = !PPX_FALSE
} ppx_packet_status_t;

/* define ppx packet format type */
typedef enum
{
   PPX_FMT_REGION     = 0x01,
   PPX_FMT_IAP        = 0x02,
} ppx_packet_format_t;

/* define ppx protocl id data type */
typedef enum
{
```

```
    PPX_ID_RSVD       = 0x00,
    PPX_ID_CCB        = 0x10,
    PPX_ID_MCB        = 0x20,
    PPX_ID_MCB_1      = 0x21,
    PPX_ID_MCB_2      = 0x22,
    PPX_ID_FCB        = 0x30,
    PPX_ID_FCB_1      = 0x31,
    PPX_ID_FCB_2      = 0x32,
    PPX_ID_BMS        = 0x40,
    PPX_ID_GPRS       = 0x50,
    PPX_ID_BLE        = 0x60,
    PPX_ID_ALARM      = 0x70,
    PPX_ID_VOICE      = 0x80,
    PPX_ID_MAX        = 0x90,
} ppx_packet_id_t;

/* define ppx packet command type */
typedef enum
{
    PPX_CMD_REQ       = 0x00,
    PPX_CMD_RSP       = 0x80,
    PPX_CMD_EXCP      = 0xC0
} ppx_cmd_type_t;

/* define ppx packet command content */
typedef enum
{
    PPX_MSG_RSVD      = 0x00,
    PPX_MSG_READ      = 0x01,
    PPX_MSG_MULTREAD  = 0x02,
    PPX_MSG_WRITE     = 0x03,
```

```
    PPX_MSG_MULTWRITE  = 0x04,
    PPX_MSG_COMPARE    = 0x05,
    PPX_MSG_UPGRADE    = 0x06,
    PPX_MSG_NOTIFY     = 0x07,
    PPX_MSG_MASK       = 0x0F,
} ppx_cmd_msg_t;
```

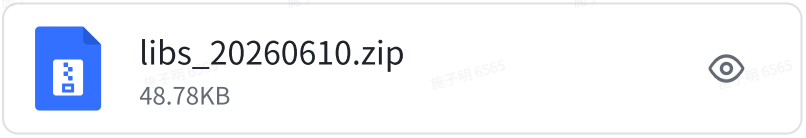
4. 协议库

协议库文件基于C编译生成，包含.a静态库及.dll动态库，C/C++可直接调用，Python调用参考ctypes的使用。

注意：64bit Python使用X64协议库，32bit Python使用X86协议库

X64

20260610



X86